



BRAINWARE UNIVERSITY

STUDENT ATTENDANCE AND RESTAURANT ORDER MANAGEMENT SYSTEM (WEEK 9)

1. Basic Information

Programs Covered:	Program 1: Student Attendance Management System Program 2: Restaurant Order Management System
Name(s) of Student(s):	Soumyadwip Das (BWU/BTS/25/169) Ankita Paul (BWU/BTS/25/180) Sonia Naskar (BWU/BTS/25/195) Joyeta Mondal (BWU/BTS/25/214) Sayantan Bharati (BWU/BTS/25/503)
Programme & Semester:	B.Tech CSE & 3rd Semester
Course Name & Code:	Python Programming Lab (BES09013)
Name of the Guide/Supervisor:	MR. PRANAB GHARAI & MR. INDRANIL DAS
Project Date	11/09/2026





2. Introduction and Background

Week 9 covers two menu-driven, terminal-based record-management tools: a Student Attendance Management System for recording daily presence/absence and computing attendance percentages, and a Restaurant Order Management System for taking orders against a fixed menu and generating a billed receipt. Both programs keep core data-handling logic in dedicated functions separate from the input/output (menu) handling, and both use in-memory Python dictionaries as their data store.

3. Objectives of the Project (Week 9)

Program 1 - Student Attendance Management System

- Add and store student records (roll, name)
- Record daily attendance status (Present/Absent) per student
- View attendance by student or by date
- Calculate attendance percentage and flag students below a threshold

Program 2 - Restaurant Order Management System

- Display a fixed restaurant menu with prices
- Take an order with multiple items and quantities
- Calculate the bill with an optional discount
- Generate a formatted receipt

4. Problem Statement / Driving Question

Program 1: How can daily attendance be recorded and summarised so low-attendance students can be identified quickly?

Program 2: How can orders against a shared menu be totalled accurately, with discounts applied correctly, and turned into a receipt?

5. Methodology (Week 9 Activities)

Program 1 - Student Attendance Management System

- Modelled each student as a dictionary entry keyed by roll number, holding a nested date-to-status dictionary
- Built core functions (add_attendance_student, record_attendance_data) with no I/O
- Built get_attendance_percentage and get_low_attendance to summarise records
- Added a parse_date helper to validate YYYY-MM-DD input
- Built UI handler functions in a menu loop that call the core functions and print boxed results

Program 2 - Restaurant Order Management System

- Modelled a fixed menu dictionary and a separate current-order dictionary
- Built core functions (add_to_order, compute_bill, build_receipt_lines) with no I/O
- Built UI handlers for displaying the menu, taking an order, and billing
- Supported an optional percentage discount applied to the subtotal
- Tested order-taking, bill calculation with and without discount, and receipt generation

6. Expected Outcomes / Deliverables (Week 9)

- Working Student Attendance Management System with percentage tracking
- Working Restaurant Order Management System with billed receipts
- Clear separation of core logic and UI code in both programs
- Accurate attendance percentage and low-attendance detection
- Accurate bill totals with correct discount handling



7. Core Logic

Program 1 - Student Attendance Management System

```
def add_attendance_student(roll, name):
    if roll in attendance_students:
        return False
    attendance_students[roll] = {'name': name, 'attendance': {}}
    return True

def record_attendance_data(roll, date_obj, status):
    if roll not in attendance_students: return False
    if status not in ('P', 'A'): return False
    attendance_students[roll]['attendance'][date_obj] = status
    return True

def get_attendance_percentage(roll):
    data = attendance_students[roll]['attendance']
    if not data: return 0.0
    present = sum(1 for s in data.values() if s == 'P')
    return (present / len(data)) * 100

def get_low_attendance(threshold):
    result = []
    for roll, data in attendance_students.items():
        att = data['attendance']
        pct = 0.0 if not att else (sum(1 for s in att.values() if s == 'P') / len(att)) * 100
        if pct < threshold:
            result.append((data['name'], roll, pct))
    return result
```

Program 2 - Restaurant Order Management System

```
def add_to_order(item, qty):
    if item not in menu: return False
    if qty is None or qty <= 0: return False
    current_order[item] = current_order.get(item, 0) + qty
    return True

def compute_bill(discount_rate=0.0):
    if not current_order: return 0.0, 0.0, 0.0
    subtotal = sum(menu[item] * qty for item, qty in current_order.items())
    discount = subtotal * discount_rate
    total = subtotal - discount
    return subtotal, discount, total

def build_receipt_lines(subtotal, discount, total, discount_rate):
    lines = ["RECEIPT:", "", f" Date: {datetime.datetime.now():%Y-%m-%d %H:%M:%S}"]
    for item, qty in current_order.items():
        price = menu[item]
        lines.append(f" {item:<12}{qty:<6}${price:<9.2f}${price*qty:.2f}")
    lines.append(f" Subtotal: ${subtotal:.2f}")
    if discount > 0:
        lines.append(f" Discount ({discount_rate*100:.0f}%): -${discount:.2f}")
    lines.append(f" Total: ${total:.2f}")
    return lines
```



8. Output Screenshot

Program 1 - Student Attendance Management System

ATTENDANCE

Terminal Student Attendance Management System

```
[1] Add Student
[2] Record Attendance
[3] View Attendance
[4] Calculate Attendance Percentage
[5] Identify Low Attendance Students
[6] Exit
```

```
> Enter option: 1
> Enter student roll number: 001
> Enter student name: ram
```

STUDENT ADDED:

```
Name : ram
Roll : 001
```

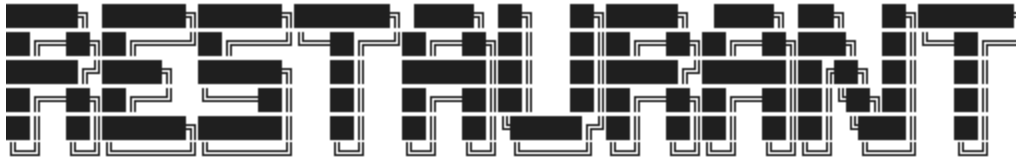
```
[1] Add Student
[2] Record Attendance
[3] View Attendance
[4] Calculate Attendance Percentage
[5] Identify Low Attendance Students
[6] Exit
```

```
> Enter option: 6
```

Goodbye!



Program 2 - Restaurant Order Management System



Terminal Restaurant Order Management System

```
[1] Display Menu
[2] Take Order
[3] Calculate Bill
[4] Generate Receipt
[5] Exit
```

> Enter option: 2

```
Enter item name (or 'done' to finish).
```

> Item (or 'done'): pizza
> Qty for Pizza: 55

```
55 x Pizza added.
```

> Item (or 'done'): done

```
ORDER TAKEN:

55 x Pizza
```

```
[1] Display Menu
[2] Take Order
[3] Calculate Bill
[4] Generate Receipt
[5] Exit
```

> Enter option: 3
> Apply discount? (yes/no): yes
> Discount percentage (0-100): 20

```
BILL:

Subtotal: $687.50
Discount (20%): -$137.50
Total Bill: $550.00
```



9. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core language for both console systems
Python Dictionaries	In-memory storage for students, attendance records, menu, and orders
Core/UI Separation	Business logic functions kept separate from input/print handlers
datetime module	Date parsing/validation and receipt timestamps
String Formatting (f-strings)	Aligned tabular display of attendance and receipts

10. References

- Python Official Documentation: <https://docs.python.org/3/>
- Python datetime Module Guide: <https://docs.python.org/3/library/datetime.html>
- Brainware University – Python Course Guidelines



11. Signatures

Signature of Student(s):

Date of Submission:

Signature of Guide/Supervisor:
